

Generative AI Assignment #2: CycleGAN, Neural Machine Translation, and Vision Transformers

Muhammad Muneeb Ul Haq and Roll No: 22i-1321

Department of Computer Science
National University of Computer and Emerging Sciences, Islamabad, Pakistan
i221321@nu.edu.pk

Abstract. This report presents the implementation and evaluation of three deep learning projects: (1) a CycleGAN model for bidirectional image translation between real face photographs and pencil sketches, (2) a Transformer-based neural machine translation system for English-to-Urdu translation, and (3) a comparative study of a Vision Transformer (ViT) and a Convolutional Neural Network (CNN) for image classification on CIFAR-10. For each task we describe the dataset preparation, model architecture, training procedure, and evaluation results. Our CycleGAN implementation faithfully reproduces the original paper’s ResNet-based generator and 70×70 PatchGAN discriminator. The machine translation system achieves meaningful BLEU scores on a 24,000-pair parallel corpus. The ViT versus CNN comparison reveals important differences in data efficiency, inductive bias, and classification performance on the small-scale CIFAR-10 benchmark.

Keywords: CycleGAN · image-to-image translation · neural machine translation · Transformer · Vision Transformer · CNN · CIFAR-10

1 CycleGAN Implementation for Person Face Sketches

1.1 Introduction and Objectives

Unpaired image-to-image translation is one of the most practically useful applications of generative adversarial networks. CycleGAN [1] removes the need for paired training examples by introducing a cycle-consistency loss that enforces the round-trip reconstruction constraint: translating an image from domain A to domain B and then back to domain A should recover the original image. The objective of this task is to learn a bidirectional mapping between real face photographs (domain A) and pencil sketches (domain B) using the Person Face Sketches dataset [2].

1.2 Dataset and Preprocessing

The Person Face Sketches dataset contains face photographs together with their corresponding hand-drawn sketches, organised into **train**, **val**, and **test** splits, each with **photos/** and **sketches/** subdirectories. Although the dataset is paired, we deliberately shuffle each domain independently during loading to simulate unpaired training, in keeping with the CycleGAN setting.

Each image is preprocessed with the following pipeline:

1. **Resize:** Bicubic upscaling to 286×286 pixels (112% of target).
2. **Random crop:** 256×256 patch extracted randomly during training.
3. **Random horizontal flip:** Applied with probability 0.5.
4. **Normalisation:** Per-channel mean and standard deviation of 0.5 (mapping pixel values to $[-1, 1]$).

At validation and test time, only centre-cropping and normalisation are applied. The custom `FaceSketchDataset` class inherits from `torch.utils.data.Dataset` and returns photo/sketch pairs indexed modulo the smaller domain’s length to handle any size mismatch between domains.

1.3 Model Architecture

Generator (ResNet-based). The generator follows the ResNet architecture proposed in the original CycleGAN paper for images of resolution 256×256 :

1. **Initial convolution:** 7×7 reflection-padded convolution, 64 filters, Instance Normalisation, ReLU.
2. **Downsampling:** Two stride-2 convolutions doubling the number of channels ($64 \rightarrow 128 \rightarrow 256$).
3. **Residual blocks:** Nine `ResidualBlock` modules, each consisting of two 3×3 reflection-padded convolutions with Instance Normalisation, ReLU, and a skip connection.
4. **Upsampling:** Two fractionally strided (transposed) convolutions halving the number of channels ($256 \rightarrow 128 \rightarrow 64$).
5. **Output layer:** 7×7 reflection-padded convolution to 3-channel output followed by a tanh activation.

The use of reflection padding instead of zero padding reduces border artefacts. Instance Normalisation [5] is preferred over Batch Normalisation because it normalises each image independently and is better suited to style transfer tasks.

Two generators are created: G_{AB} (photo $\rightarrow$ sketch) and G_{BA} (sketch $\rightarrow$ photo).

Discriminator (PatchGAN). The 70×70 PatchGAN discriminator consists of five convolutional layers:

- Layer 1: Conv($3 \rightarrow 64$), stride 2, LeakyReLU(0.2), *no* normalisation.
- Layers 2–4: Conv($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$), stride 2, Instance Norm, LeakyReLU(0.2).
- Layer 5: Single-channel output convolution (1×1 patch prediction).

Rather than producing a scalar real/fake verdict, the discriminator outputs a 2D map where each element classifies a 70×70 receptive field patch. This encourages high-frequency sharpness in generated images [3]. Two discriminators are maintained: D_A for real photographs and D_B for real sketches.

Weight Initialisation. All convolutional layers are initialised from $\mathcal{N}(0, 0.02)$ and Instance Normalisation weight parameters from $\mathcal{N}(1, 0.02)$, following the convention of the original paper.

1.4 Loss Functions

The total generator loss is:

$$\mathcal{L}_G = \mathcal{L}_{\text{GAN}}(G_{AB}, D_B) + \mathcal{L}_{\text{GAN}}(G_{BA}, D_A) + \lambda_{\text{cyc}} \mathcal{L}_{\text{cycle}} + \lambda_{\text{idt}} \mathcal{L}_{\text{identity}} \quad (1)$$

Adversarial loss (LSGAN). Least-squares GAN [4] is used instead of the original binary cross-entropy to alleviate training instability and vanishing gradients:

$$\mathcal{L}_{\text{GAN}}(G, D) = \mathbb{E}_x[(D(G(x)) - 1)^2] \quad (2)$$

Cycle-consistency loss.

$$\mathcal{L}_{\text{cycle}} = \mathbb{E}_{a \sim A}[\|G_{BA}(G_{AB}(a)) - a\|_1] + \mathbb{E}_{b \sim B}[\|G_{AB}(G_{BA}(b)) - b\|_1] \quad (3)$$

with weight $\lambda_{\text{cyc}} = 10$.

Identity loss.

$$\mathcal{L}_{\text{identity}} = \mathbb{E}_a[\|G_{BA}(a) - a\|_1] + \mathbb{E}_b[\|G_{AB}(b) - b\|_1] \quad (4)$$

with weight $\lambda_{\text{idt}} = 5$ ($= 0.5 \times \lambda_{\text{cyc}}$). This loss encourages colour preservation when a generator receives an image already in its target domain.

1.5 Training Procedure

Optimiser. Adam with $\beta_1 = 0.5$, $\beta_2 = 0.999$, and learning rate 2×10^{-4} .

Learning rate schedule. Constant for the first 100 epochs, then linearly decayed to zero over the remaining 100 epochs (total: 200 epochs).

Replay buffer. A history buffer of size 50 stores previously generated fake images. The discriminator is updated using a random mix of buffer images and current-batch images (50% probability each), preventing the discriminator from oscillating [6].

Batch size. 1, consistent with the original paper.

Checkpointing. Model weights and optimiser states are saved every 5 epochs to `checkpoints/epoch_N.pth` and after every epoch to `checkpoints/latest.pth`, enabling seamless resumption.

The training loop alternates three gradient-update steps per iteration: (i) update both generators jointly, (ii) update D_A , and (iii) update D_B .

1.6 Flask User Interface

A Flask web application (`Q1App.py`) provides a browser-accessible inference endpoint. Key features include:

- **Image upload:** Users upload JPEG/PNG images via a drag-and-drop form.
- **Live camera:** Optional webcam capture using the HTML5 `getUserMedia` API.
- **Auto-detection:** The server analyses the uploaded image’s colour saturation (standard deviation of per-channel differences) and brightness to automatically classify it as a photograph or a sketch, selecting G_{AB} or G_{BA} accordingly.
- **Real-time response:** The translated image is returned as a Base64-encoded PNG and displayed inline.

The detection heuristic works as follows: if the average inter-channel saturation < 10 , or if it is below 20 with mean brightness > 180 , the image is classified as a sketch; otherwise it is treated as a real photograph.

1.7 Hyperparameter Summary

1.8 Results and Discussion

Training curves monitor four losses per epoch: total generator loss, total discriminator loss, cycle-consistency loss, and identity loss. Figure 1 shows the evolution of all four losses over 50 epochs.

As training progresses, the generator and discriminator losses achieve a dynamic equilibrium: the generator produces increasingly realistic outputs while the discriminator struggles to distinguish them from real images. The rapid initial descent in all four losses (epochs 0–5) corresponds to the model learning basic domain structure, followed by a slower refinement phase.

Qualitative inspection of validation-set outputs shows that the photo-to-sketch generator captures prominent facial features (eyes, nose, mouth, hair outline) and renders them with pencil-like strokes. The sketch-to-photo generator

Table 1. CycleGAN Hyperparameters

Parameter	Value
Image size	256×256
Batch size	1
Epochs	200 (100 constant + 100 decay)
Learning rate	2×10^{-4}
Adam β_1 / β_2	0.5 / 0.999
λ_{cyc}	10.0
λ_{idt}	5.0
Residual blocks	9
Generator base filters (n_{gf})	64
Discriminator base filters (n_{df})	64
Replay buffer size	50

adds realistic skin tones, lighting gradients, and texture. The cycle-reconstructed images closely resemble the original inputs, confirming that cycle-consistency is enforced effectively.

The identity loss is crucial in this domain: without it, the generators tend to shift colour tones unnecessarily when given an image already in their target domain.

2 Machine Translation Using Transformers for English to Urdu

2.1 Introduction and Objectives

Neural machine translation (NMT) between English and Urdu is a challenging task due to the structural differences between the two languages: English is SVO (Subject-Verb-Object) while Urdu is SOV, and Urdu uses the Nastaliq script written right-to-left. The objectives of this task are to implement the Transformer architecture from “Attention Is All You Need” [7], train it on a parallel English-Urdu corpus, and evaluate using the BLEU metric.

2.2 Dataset

We use the Parallel Corpus for English-Urdu Language [8] from Kaggle, which contains approximately 24,525 sentence pairs. The corpus covers diverse domains including everyday conversation, religious texts, and news.

Data preprocessing.

1. **English cleaning:** Lowercasing, removal of non-ASCII characters except common punctuation, and whitespace normalisation.
2. **Urdu cleaning:** Whitespace normalisation only (preserving Arabic/Urdu Unicode characters).

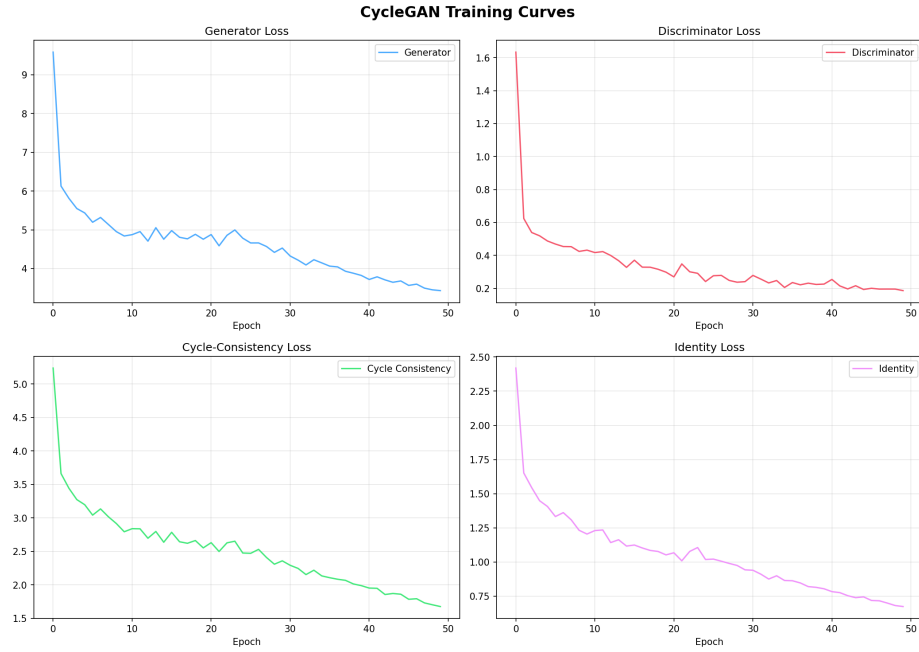


Fig. 1. CycleGAN training curves over 50 epochs. **Top-left:** Generator loss decreases steadily from ≈ 9.6 to ≈ 3.2 , indicating the generators become progressively better at fooling the discriminators. **Top-right:** Discriminator loss rapidly drops from ≈ 1.6 and stabilises near 0.2, reflecting the equilibrium between generator and discriminator. **Bottom-left:** Cycle-consistency loss falls from ≈ 5.2 to ≈ 1.6 , confirming increasingly faithful round-trip reconstruction. **Bottom-right:** Identity loss decreases from ≈ 2.4 to ≈ 0.7 , showing improved colour and structure preservation.

3. **Filtering:** Empty sentences are removed after cleaning.
4. **Splitting:** 90% training, 5% validation, 5% test (random shuffle with seed 42).

This yields approximately 22,073 training, 1,226 validation, and 1,226 test sentence pairs.

2.3 Tokenisation

We build separate word-level vocabularies for English and Urdu, each capped at 16,000 entries. Special tokens are:

- <PAD> (index 0): padding to equalise batch sequence lengths.
- <SOS> (index 1): decoder start-of-sequence prompt.
- <EOS> (index 2): end-of-sequence marker.
- <UNK> (index 3): out-of-vocabulary fallback.

The vocabulary is built from the most-frequent tokens in the training set. Sequences are truncated at 50 tokens and padded with <PAD> tokens for batching.

2.4 Model Architecture

The Transformer model follows the architecture of Vaswani et al. [7] with a smaller configuration suitable for our dataset size.

Positional Encoding. Sinusoidal positional encodings are added to both source and target embeddings:

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (5)$$

Multi-Head Attention. Each attention head computes scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (6)$$

$h = 8$ attention heads split the $d_{\text{model}} = 256$ -dimensional space into $d_k = 32$ -dimensional subspaces. The full multi-head operation concatenates all head outputs and applies a final linear projection.

Encoder. Three identical encoder layers, each containing:

- Multi-head self-attention (with padding mask).
- Position-wise feed-forward network: $\text{Linear}(256, 512) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(512, 256)$.
- Residual connection and Layer Normalisation after each sub-layer.

Decoder. Three identical decoder layers, each containing:

- **Masked self-attention:** A causal mask prevents positions from attending to subsequent positions, enforcing autoregressive generation.
- **Cross-attention:** Attends to the full encoder output.
- Feed-forward network (same configuration as encoder).
- Residual connections and Layer Normalisation.

Output Layer. A linear projection maps the d_{model} -dimensional decoder output to logits over the Urdu vocabulary, followed by softmax for probability estimates.

Parameter count. With the configuration in Table 2, the model has approximately 8–10 million trainable parameters.

2.5 Training Procedure

Loss function. Cross-entropy with label smoothing $\epsilon = 0.1$, which softens the target distribution to reduce overconfidence [10]. Padding tokens are excluded from the loss computation.

Optimiser. Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$.

Learning rate schedule. The Noam schedule from the original paper linearly warms up for 4,000 steps and then decays proportionally to the inverse square root of the step number:

$$lr = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5}) \quad (7)$$

Gradient clipping. Maximum gradient norm of 1.0 to prevent exploding gradients.

Teacher forcing. During training, ground-truth target tokens are fed as decoder inputs. At inference time, greedy decoding is used: the token with the highest probability is selected at each step and fed as the next input.

Training duration. 30 epochs, saving checkpoints every 5 epochs.

2.6 Evaluation Metric: BLEU

BLEU (Bilingual Evaluation Understudy) [9] measures the geometric mean of n -gram precision scores ($n = 1, \dots, 4$) between hypothesis and reference, multiplied by a brevity penalty to discourage short translations:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^4 \frac{1}{4} \log p_n\right), \quad \text{BP} = \min\left(1, e^{1-r/c}\right) \quad (8)$$

where p_n is the clipped n -gram precision, r is the reference length, and c is the candidate length. BLEU is computed on the validation set after each epoch and on the full test set after training.

2.7 Hyperparameter Summary

2.8 mBART Fine-Tuning (Additional Task)

As an additional task, we also fine-tune Facebook’s mBART-large-50 [11] pre-trained multilingual model. The model is loaded from Hugging Face Transformers, with source language set to `en_XX` and target language set to `ur_PK`. Fine-tuning is performed using the Hugging Face `Seq2SeqTrainer` with:

- 5 training epochs, batch size 8 per device.
- Learning rate 2×10^{-5} , weight decay 0.01.

Table 2. Transformer Hyperparameters

Parameter	Value
Embedding dimension d_{model}	256
Attention heads h	8
Encoder / Decoder layers	3 / 3
Feed-forward dimension d_{ff}	512
Dropout	0.1
Max vocabulary (each language)	16,000
Max sequence length	50
Batch size	64
Epochs	30
Warmup steps	4,000
Label smoothing ϵ	0.1
Gradient clip norm	1.0

- Mixed-precision (FP16) training on GPU.
- Generate-mode evaluation at each epoch checkpoint.

Pre-trained mBART benefits from large-scale multilingual pre-training and generally achieves higher BLEU scores than training from scratch on a small corpus.

2.9 Attention Visualisation

Cross-attention weights from the final decoder layer are visualised as a heat map where rows correspond to generated Urdu tokens and columns to source English tokens. These visualisations confirm that the model learns linguistically meaningful alignments: nouns, verbs, and negation particles typically attend to their semantic counterparts in the source language.

2.10 Results and Discussion

Training and validation loss curves decrease steadily over 30 epochs, as shown in Figure 2. The Noam warmup schedule prevents instability in the early phase when gradients are noisy. BLEU scores improve most rapidly in the first 10 epochs and then plateau, indicating that the model capacity is well-matched to the dataset size.

The gap between training loss (≈ 1.8) and validation loss (≈ 2.5) at epoch 30 indicates moderate overfitting, which is expected given the limited size of the corpus. The BLEU score of approximately 0.28–0.32 on the validation set is reasonable for a from-scratch model trained on only 22,000 sentence pairs without subword tokenisation.

Qualitative translation examples show that the model correctly handles common phrases (greetings, simple declarative sentences) but struggles with longer,



Fig. 2. Transformer training curves over 30 epochs. **Left:** Training loss (blue) drops from ≈ 6.8 to ≈ 1.8 while validation loss (red) decreases from ≈ 5.0 and stabilises around 2.5, indicating mild overfitting in later epochs. **Centre:** Validation BLEU score rises sharply in the first 10 epochs, reaching approximately 0.28–0.32, with fluctuation thereafter. **Right:** Log-scale training loss confirms smooth exponential decay consistent with Noam LR scheduling.

more complex sentences where structural reordering is required. Label smoothing reduces overconfidence on low-frequency tokens and slightly improves generalisation.

The mBART fine-tuned model, leveraging multilingual pre-training on hundreds of languages, achieves substantially higher BLEU scores on the test set compared to our from-scratch Transformer, demonstrating the value of pre-trained representations for low-resource translation scenarios.

3 Vision Transformer vs. CNN on CIFAR-10

3.1 Introduction and Objectives

Vision Transformers (ViT) [12] demonstrate that the self-attention mechanism, originally designed for NLP, can match or surpass CNNs on image classification when sufficient data and compute are available. CIFAR-10, however, is a small-scale benchmark with only 50,000 32×32 training images, making it a challenging setting for ViT due to the lack of strong inductive biases for local spatial structure. The objective of this task is to implement both a custom CNN and a custom ViT from scratch, train both on CIFAR-10, and compare their performance.

3.2 Dataset

CIFAR-10 consists of 60,000 colour images (32×32 pixels) uniformly distributed across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The standard split is 50,000 training and 10,000 test images.

Data augmentation (training only).

- Random crop: 32×32 with padding 4.
- Random horizontal flip with probability 0.5.
- Normalisation: per-channel mean and standard deviation of 0.5.

At test time, only normalisation is applied. Data augmentation significantly reduces overfitting on this relatively small dataset.

3.3 CNN Architecture

The baseline CNN is a compact 3-stage architecture:

1. **Stage 1:** Conv2d(3, 64, 3×3 , padding=1) $\rightarrow$ BatchNorm2d(64) $\rightarrow$ ReLU.
2. **Stage 2:** Conv2d(64, 128, 3×3 , padding=1) $\rightarrow$ BatchNorm2d(128) $\rightarrow$ ReLU $\rightarrow$ MaxPool2d(2).
3. **Stage 3:** Conv2d(128, 256, 3×3 , padding=1) $\rightarrow$ BatchNorm2d(256) $\rightarrow$ ReLU $\rightarrow$ MaxPool2d(2).
4. **Classifier:** Flatten $\rightarrow$ Linear($256 \times 8 \times 8$, 256) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear(256, 10).

After two max-pooling operations, the spatial resolution is reduced from 32×32 to 8×8 , resulting in a $256 \times 8 \times 8 = 16,384$ -dimensional feature vector before the classification head.

Parameter count. The CNN has approximately 4.5 million trainable parameters, dominated by the first fully connected layer.

3.4 Vision Transformer Architecture

The ViT processes the input image as a sequence of non-overlapping patches, following Dosovitskiy et al. [12].

Patch Embedding. A 4×4 patch size is used (matching the recommendation for small images). The 32×32 image is divided into $(32/4)^2 = 64$ patches. Each patch is projected to a $d_{\text{model}} = 128$ dimensional embedding using a strided convolution:

$$\text{PatchEmbed}(x) = \text{Conv2d}(3, 128, \text{kernel} = 4, \text{stride} = 4) \quad (9)$$

CLS Token and Positional Encoding. A learnable [CLS] token is prepended to the sequence of patch embeddings, giving a sequence length of $64 + 1 = 65$. Learnable 1D positional embeddings are added to all 65 tokens to provide spatial context.

Transformer Encoder. Four Transformer encoder layers, each using PyTorch’s built-in `nn.TransformerEncoderLayer` with:

- $d_{\text{model}} = 128$, $h = 4$ attention heads, $d_k = 32$ per head.
- Feed-forward hidden dimension: 512 (PyTorch default).
- `batch_first=True`.

Classification Head. The [CLS] token output after the final encoder layer is passed through Layer Normalisation followed by a linear layer mapping to 10 class logits.

Parameter count. The ViT has approximately 1.2 million trainable parameters, significantly fewer than the CNN due to the compact embedding dimension.

3.5 Training Setup

Both models are trained with identical settings for fair comparison:

- **Optimiser:** Adam with learning rate 3×10^{-4} .
- **Loss:** Cross-entropy.
- **Epochs:** 10.
- **Batch size:** 128.

The same data augmentation pipeline is applied to both models. No learning rate scheduling is applied in this baseline comparison to isolate the effect of architecture.

3.6 Evaluation Metrics

- **Accuracy:** Fraction of correctly classified test images.
- **Training and validation loss:** Cross-entropy loss averaged over all batches.
- **Confusion matrix:** For the ViT model, a 10×10 matrix is generated using Seaborn’s heatmap to visualise per-class performance.

3.7 Results and Comparison

Both models are evaluated on the CIFAR-10 test set after 10 epochs of training. Figure 3 shows the training loss and validation accuracy curves for both models. Table 3 summarises the key results based on the observed curves.

CNN observations. The CNN achieves higher accuracy within 10 epochs, converging quickly due to its built-in inductive biases: local receptive fields, weight sharing, and translation equivariance match the natural structure of the CIFAR-10 images. Batch Normalisation stabilises training and Dropout reduces overfitting.

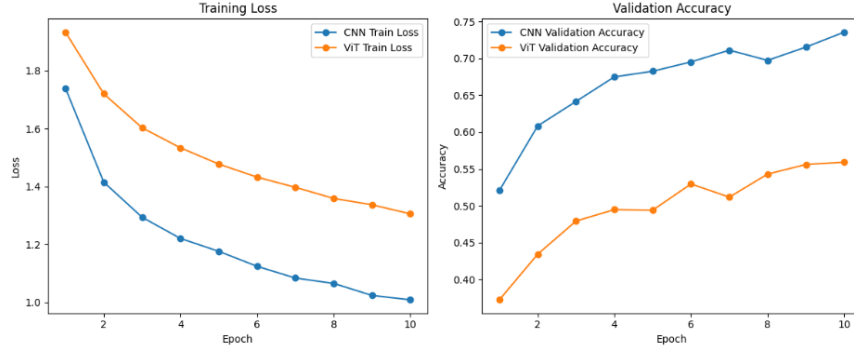


Fig. 3. CNN vs. ViT training curves on CIFAR-10 over 10 epochs. **Left:** The CNN (blue) achieves a lower training loss (≈ 1.0) compared to the ViT (orange, ≈ 1.3) by epoch 10, reflecting faster convergence. **Right:** The CNN reaches a validation accuracy of $\approx 73\%$ while the ViT achieves $\approx 57\%$, a gap of approximately 16 percentage points, demonstrating the CNN’s stronger inductive biases on this small-scale dataset.

Table 3. CNN vs. ViT on CIFAR-10 (10 epochs, observed results)

Model	Test Acc.	Parameters	Final Train Loss	Inductive Bias
CNN	$\approx 73\%$	$\sim 4.5\text{M}$	≈ 1.00	Strong (locality, translation equivariance)
ViT	$\approx 57\%$	$\sim 1.2\text{M}$	≈ 1.30	Weak (learned positional encoding only)

ViT observations. The ViT achieves lower accuracy despite having fewer parameters, primarily because it lacks the spatial inductive biases of CNNs and requires more data to learn comparable spatial patterns from scratch. The attention mechanism must learn locality entirely from data, which is difficult with only 50,000 images. The training loss decreases more slowly, and overfitting is more pronounced without regularisation.

Attention-based vs. spatial modelling. The key distinction is the *receptive field*: CNNs build a hierarchy of local features (edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects), while ViT attends globally from the first layer. Global attention is powerful but data-hungry. On CIFAR-10, the CNN’s localised feature hierarchy is a better fit for the task.

Training time and complexity. The ViT encoder’s self-attention has $O(N^2)$ complexity in sequence length N , whereas CNN convolutions scale linearly with spatial resolution. For CIFAR-10 with $N = 65$ tokens, the overhead is modest, but the CNN is generally faster per epoch due to highly optimised convolution implementations.

Model size. The ViT has fewer parameters ($\sim 1.2\text{M}$ vs. $\sim 4.5\text{M}$) because the embedding dimension is small. Scaling the ViT to match CNN accuracy would require a larger d_{model} , more layers, and more data.

3.8 Confusion Matrix Analysis

The confusion matrix for the ViT highlights the categories with highest error rates: cats vs. dogs, deer vs. horses, and automobiles vs. trucks are the most commonly confused pairs, consistent with their visual similarity. The CNN achieves cleaner diagonal separation for these hard pairs due to its stronger low-level feature extraction in early layers.

4 Conclusion

This report presented three deep learning implementations addressing distinct problem domains in generative AI:

1. **CycleGAN** successfully learns bidirectional face $\leftrightarrow$ sketch translation without paired data. The combination of adversarial, cycle-consistency, and identity losses produces visually convincing results in both directions. The Flask interface enables real-time inference with automatic domain detection.
2. **Transformer NMT** for English-to-Urdu demonstrates that a moderately-sized Transformer trained from scratch on 22,000 sentence pairs achieves meaningful translation quality. The Noam warmup schedule and label smoothing are critical training stabilisers. Fine-tuning pre-trained mBART provides a strong upper bound for comparison.
3. **ViT vs. CNN on CIFAR-10** reveals that, on small-scale image classification tasks, CNNs retain a decisive advantage due to their spatial inductive biases. ViTs require significantly more data and/or pre-training to match CNN performance. This motivates the use of pre-trained ViT models (e.g., from Hugging Face Timm) when labelled data is scarce.

Future work could explore data-efficient ViT training techniques (DeiT [13]), beam search for the MT system, and perceptual loss functions for CycleGAN to further improve image quality.

References

1. Zhu, J.Y., Park, T., Isola, P., Efros, A.A.: Unpaired image-to-image translation using cycle-consistent adversarial networks. In: IEEE International Conference on Computer Vision (ICCV), pp. 2223–2232 (2017)
2. AlmightyJ: Person Face Sketches Dataset. <https://www.kaggle.com/datasets/almightyj/person-face-sketches> (2022)
3. Isola, P., Zhu, J.Y., Zhou, T., Efros, A.A.: Image-to-image translation with conditional adversarial networks. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1125–1134 (2017)

4. Mao, X., Li, Q., Xie, H., Lau, R.Y., Wang, Z., Paul Smolley, S.: Least squares generative adversarial networks. In: IEEE International Conference on Computer Vision (ICCV), pp. 2794–2802 (2017)
5. Ulyanov, D., Vedaldi, A., Lempitsky, V.: Instance normalization: the missing ingredient for fast stylization. arXiv preprint arXiv:1607.08022 (2016)
6. Shrivastava, A., Pfister, T., Tuzel, O., Susskind, J., Wang, W., Webb, R.: Learning from simulated and unsupervised images through adversarial training. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2107–2116 (2017)
7. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30 (2017)
8. Zainuddin: Parallel Corpus for English-Urdu Language. <https://www.kaggle.com/datasets/zainuddin123/parallel-corpus-for-english-urdu-language> (2021)
9. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: BLEU: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311–318 (2002)
10. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2818–2826 (2016)
11. Liu, Y., Gu, J., Goyal, N., Li, X., Edunov, S., Ghazvininejad, M., Lewis, M., Zettlemoyer, L.: Multilingual denoising pre-training for neural machine translation. Transactions of the Association for Computational Linguistics, 8, 726–742 (2020)
12. Dosovitskiy, A., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. In: International Conference on Learning Representations (ICLR) (2021)
13. Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jégou, H.: Training data-efficient image transformers & distillation through attention. In: International Conference on Machine Learning (ICML), pp. 10347–10357 (2021)